

LLM-Augmented Log Anomaly Detection: Automated Root Cause Narration for AIOps using Lightweight Foundation Models on Real-World Campus Infrastructure

Trinh Hoang Tu¹ and Cao Tien Thanh^{1*}

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages - Information Technology,
Ho Chi Minh City, Vietnam
tth.csec2005@gmail.com, thanhct@huflit.edu.vn

Abstract. Modern enterprise and educational campus infrastructures generate massive volumes of heterogeneous, unstructured log streams across multi-site networks. While deep sequence models achieve high statistical accuracy in anomaly detection, traditional Artificial Intelligence for IT Operations (AIOps) platforms suffer from a critical explainability bottleneck: scalar reconstruction error scores fail to communicate actionable root causes, forcing Security Operations Center (SOC) analysts to manually triage thousands of log messages under severe time constraints. In this paper, we propose an end-to-end **LLM-Augmented AIOps Framework** synthesizing streaming Drain3 log template parsing, a Masked Temporal Convolutional Network and Transformer (TCN-Transformer) Autoencoder, and a lightweight 4-bit quantized Foundation Model (Phi-3 Mini in NF4 / Mistral-7B in AWQ INT4 with RAG-guided prompting) for real-time automated Root Cause Analysis (RCA) narration. Evaluated on a production 19.28 GB enterprise campus log dataset (14,850,220 log lines) from HUFLIT alongside standard benchmarks (BGL, HDFS, and UNSW-NB15) under strict chronological partitioning across 5 random seeds, our hybrid masked detector achieves an anomaly detection F1-score of 0.9521 ± 0.004 and ROC-AUC of 0.9784 ± 0.003 on campus logs, outperforming selected representative baselines (DeepLog, LogAnomaly, and Transformer AE) while executing single-sequence inference in 0.1316 ms (0.0128 ms vectorized GPU batch-64). Furthermore, our automated RAG RCA pipeline synthesizes structured incident playbooks in 880.5 ms (with 874.5 ms LLM generation, providing rapid preliminary triage compared to 15.4 ± 2.8 minutes manual investigation) with high reference-text similarity (ROUGE-L 0.8420, BLEU-4 0.7815) and variant-blind expert correctness of 4.76/5.0 with 0/50 unsafe recommendations observed.

Keywords: AIOps · Log Anomaly Detection · Large Language Models · Root Cause Analysis · TCN-Transformer Autoencoder · Drain3 · Enterprise Campus Infrastructure.

* Corresponding author: thanhct@huflit.edu.vn

1 Introduction

Enterprise networks and higher education IT infrastructures generate high-density syslog telemetry continuously streaming from perimeter firewalls, Active Directory domain controllers, Learning Management Systems (LMS), student authentication portals, and core routing backbones [1, 2]. Rapid and accurate anomaly detection within these log streams is paramount for thwarting lateral cyber attacks, credential brute-forcing, data exfiltration, and service disruptions [3].

Recent advancements in deep learning have introduced sequence-based log anomaly detection models, notably DeepLog (LSTM-based next-token prediction) [2], LogAnomaly (semantic template vectors) [3], LogRobust [4], and SwissLog [5]. While these architectures achieve strong empirical detection fidelity on synthetic or curated benchmarks, existing AIOps paradigms exhibit two fundamental operational bottlenecks in production environments:

1. **Explainability Deficit:** Contemporary anomaly detectors produce scalar anomaly scores or probabilistic reconstruction errors (e.g., $RE = 0.942$). These numerical scores provide zero contextual explanation regarding *why* a sequence is anomalous, *what* specific attack vector or hardware malfunction is occurring, or *which* downstream services are impacted.
2. **Prolonged Incident Triage & SOC Alert Fatigue:** Security Operations Center (SOC) analysts and network reliability engineers must manually query raw distributed log stores, correlate timestamps across multi-tier appliances, and formulate remediation scripts. In enterprise campus settings, this manual triage process frequently requires 15–30 minutes per critical incident, allowing fast-propagating threats (such as automated credential stuffing or spanning tree broadcast loops) to cause widespread disruption.

To overcome these operational challenges, recent 2025–2026 literature, such as the comprehensive LLM4Log survey (reviewing 145 studies) [19] and the hierarchical KRONE framework (ICDE 2026) [20], have emphasized the paradigm shift toward integrating Large Language Models (LLMs) for automated log summarization and diagnostics. However, many operator-facing LLM RCA systems rely on external cloud APIs or exceed edge memory budgets, inducing inference latency and telemetry privacy overheads.

In this paper, we present an end-to-end **LLM-Augmented AIOps Framework** tailored specifically for production enterprise campus environments. By synthesizing online Drain3 parsing, a Masked TCN-Transformer Autoencoder, and a local 4-bit quantized Foundation Model with Retrieval-Augmented Generation (RAG), our architecture automatically produces natural language Root Cause Analysis (RCA) narratives and structured incident mitigation playbooks within milliseconds of anomaly detection.

The core research contributions of this paper are summarized as follows:

- **End-to-End AIOps Architecture:** We construct a unified 3-stage pipeline integrating online Drain3 template extraction, a masked TCN-Transformer autoencoder for microsecond-level sequence anomaly scoring, and an LLM-driven narration engine.



Fig. 1. Architectural Overview of the LLM-Augmented AIOps Framework: (1) Streaming Drain3 log parsing and template encoding; (2) Masked TCN-Transformer autoencoder sequence scoring; and (3) RAG-Grounded Quantized Foundation Model Root Cause Narration Engine.

- **Masked TCN-Transformer Sequence Autoencoder:** We formulate a masked sequence modeling objective (15% dynamic token masking) that prevents trivial identity copying, combining 1D dilated causal convolutions (capturing local short-term temporal dependencies) with Multi-Head Self-Attention (capturing long-range contextual correlations across log templates), achieving superior detection accuracy with sub-millisecond single-sequence inference (0.1316 ms CPU / 0.0128 ms GPU).
- **RAG-Grounded Foundation Model RCA Narration:** We formulate a Retrieval-Augmented Generation (RAG) prompting architecture using a local FAISS index of institutional Standard Operating Procedures (SOPs) and 4-bit quantized open-source foundation models (Microsoft Phi-3 Mini in NF4 / Mistral-7B in AWQ INT4) to synthesize structured, human-readable RCA narratives and step-by-step mitigation playbooks.
- **Empirical Validation on 19.28 GB Real-World Production Logs:** We validate our framework on a comprehensive 19.28 GB enterprise campus log dataset (14,850,220 log lines) collected from the HUFLIT production infrastructure, alongside standard public benchmarks (BGL, HDFS, and UNSW-NB15), demonstrating an anomaly detection F1-score of 0.9521 ± 0.004 , ROC-AUC of 0.9784 ± 0.003 , outperforming representative baselines under our evaluation protocol with rigorous variant-blind expert validation (4.76/5.0 correctness score with 0/50 unsafe commands observed).

2 Related Work

2.1 Log Parsing and Template Extraction

Log parsing transforms unstructured, free-form text messages into structured event templates by separating constant message strings from dynamic parameters (e.g., IP addresses, ports, timestamps) [1]. Early approaches employed rule-based parsing, fixed-depth parse trees, or streaming sequence-matching techniques [1, 18]. He et al. [1] proposed Drain, an online parser using a fixed-depth parse tree that achieves high parsing accuracy and throughput. Drain3 extends Drain with persistent caching and streaming support. In this work, we employ

Drain3 to sanitize high-throughput syslog streams and generate discrete template identifiers for downstream sequence modeling.

2.2 Deep Learning for Log Anomaly Detection

Sequence-based anomaly detection models learn temporal transition dynamics over sequences of log templates [2, 3]. Du et al. [2] introduced DeepLog, employing Long Short-Term Memory (LSTM) networks to model normal log sequences and flag unexpected subsequent templates as anomalous. Meng et al. [3] proposed LogAnomaly, integrating semantic template vectors via FastText to capture semantic similarities. SwissLog [5] and LogRobust [4] further address log instability and interleaved execution threads. While deep sequence architectures achieve strong detection precision, they operate as black-box scoring engines: scalar anomaly metrics fail to explain the operational root cause, imposing heavy manual triage overhead on operators.

2.3 Large Language Models in Log Analytics and RCA

Recent 2025–2026 surveys, notably LLM4Log [19], categorize emerging LLM paradigms in log analytics, identifying key patterns including parser-free pre-training, retrieval-augmented grounding, and modular verifiers. Ma et al. [20] proposed KRONE (ICDE 2026), a hierarchical framework utilizing nested LLM modules for log anomaly detection. MaidLog [21] (ICDE 2026) investigates zero-shot detection on resource-constrained systems, while LogMUSE [22] and ContraLog [23] examine multi-scale semantic embeddings and contrastive representations. In industry settings, Smart Eye [24] (WWW 2026) implements a proposer-verifier framework. Cloud incident root-cause recommendation using LLMs has also been explored by Ahmed et al. [14] and surveyed in [15]. In contrast to high-latency external LLM APIs, our framework couples microsecond TCN-Transformer anomaly filtering with edge-quantized local LLMs and RAG grounding.

Table 1. Systematic Comparison with State-of-the-Art Log Anomaly Detection Approaches

Approach	Log Parser	Sequence Detector	RCA Mechanism	Edge RCA	Detection Latency	Target Domain
DeepLog [2]	Spell / Drain	LSTM Next-Token	None (Scalar Score)	×	~ 1.25 ms	HDFS, BGL
LogAnomaly [3]	FT-Tree	FT-LSTM	None (Scalar Score)	×	~ 1.84 ms	BGL, Synthetic
LogRobust [4]	Drain	Bi-LSTM + Attn	None (Scalar Score)	×	~ 2.10 ms	Hadoop, HDFS
SwissLog [5]	Drain3	Hierarchical BERT	Heuristic Matching	×	~ 5.40 ms	OpenStack
KRONE [20]	Hierarchy	Automaton/Local	Nested LLM	×	NR	Industrial Cloud
Proposed Framework	Drain3	Masked TCN-Transf.	4-bit RAG Foundation Model	✓	0.1316 ms (CPU)	Enterprise Campus

3 Methodology

3.1 System Architecture Overview

The proposed LLM-Augmented AIOps Framework operates across three pipelined stages: (i) **Stage 1 (Online Parsing)**: Raw syslog streams are ingested and

tokenized by Drain3 into structured template IDs; (ii) **Stage 2 (Masked Sequence Anomaly Detection)**: Sliding template windows are masked, vectorized, and scored by our Masked TCN-Transformer Autoencoder; (iii) **Stage 3 (RAG-Grounded RCA Narration)**: Triggered anomalies activate the local 4-bit quantized Foundation Model to retrieve relevant institutional runbooks via FAISS and synthesize structured JSON incident reports.

3.2 Masked TCN-Transformer Denoising Sequence Model

To detect subtle temporal and distributional anomalies without allowing the model to learn trivial identity shortcuts, we formulate our detector as a **Masked Log Sequence Autoencoder (MLSA)** operating under a denoising autoencoder paradigm: input sequences are corrupted via 15% dynamic token masking, representations are extracted via a Temporal Convolutional Network (TCN) [7] and Transformer Self-Attention [8], and a linear vocabulary classification head acts as the reconstruction decoder.

Embedding and Dynamic Masking A sliding window of length $L = 20$ (with stride $S = 1$) extracts template sequences $\mathbf{X} = [T_1, T_2, \dots, T_L]$. During training, 15% of the template tokens are randomly replaced with a reserved [MASK] token ($ID = 0$). Input tokens are embedded into a d_{model} -dimensional continuous vector space:

$$\mathbf{E} = \text{Embedding}(\mathbf{X}) + \mathbf{P}_{\text{pos}} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

where $\mathbf{P}_{\text{pos}}$ denotes sinusoidal positional encodings [8]:

$$\mathbf{P}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \mathbf{P}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

Temporal Convolutional Network (TCN) Layer To capture localized temporal patterns across consecutive log events, the embedded sequence $\mathbf{E}$ passes through a stack of 1D dilated causal convolutional layers [7]. For an input sequence $\mathbf{x} \in \mathbb{R}^L$ and a convolutional filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation with dilation factor d is defined as:

$$\mathbf{y}(t) = (\mathbf{x} *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}(t - d \cdot i) \quad (3)$$

We configure dilation factors $d \in \{1, 2\}$, enforcing causal convolutions for localized temporal extraction without future feature leakage. Residual connections and weight normalization ensure stable gradient propagation:

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\mathbf{E} + \text{GELU}(\text{DilatedConv1D}(\mathbf{E}))) \quad (4)$$

Transformer Multi-Head Self-Attention Encoder The output $\mathbf{H}_{\text{TCN}}$ is then fed into a Transformer Encoder block to model long-range inter-template correlations. The encoder utilizes bidirectional self-attention by design to contextualize masked tokens using surrounding sequence events. For queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (5)$$

$$\text{MHA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (6)$$

$$\text{head}_i = \text{Attention}(\mathbf{H}\mathbf{W}_i^Q, \mathbf{H}\mathbf{W}_i^K, \mathbf{H}\mathbf{W}_i^V) \quad (7)$$

The Transformer block applies pre-LayerNorm and a two-layer Feed-Forward Network (FFN) with GELU activations:

$$\text{FFN}(\mathbf{Z}) = \max(0, \mathbf{Z}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (8)$$

$$\mathbf{Z} = \text{LayerNorm}(\mathbf{H} + \text{MHA}(\mathbf{H})) \quad (9)$$

$$\mathbf{H}_{\text{out}} = \text{LayerNorm}(\mathbf{Z} + \text{FFN}(\mathbf{Z})) \quad (10)$$

Masked Reconstruction Objective and Anomaly Scoring A linear classification head projects $\mathbf{H}_{\text{out}}$ back to the template vocabulary space $\mathbb{R}^{|\mathcal{V}|}$, serving as the reconstruction decoder. The model is trained to predict original template identities at masked positions via Cross-Entropy loss:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log P(T_i = t_i \mid \mathbf{X}_{\setminus \mathcal{M}}) \quad (11)$$

During inference, we compute the pseudo-log-likelihood anomaly score across the sequence by masking each position $t \in \{1, \dots, L\}$ in a vectorized batch:

$$S(\mathbf{X}) = \frac{1}{L} \sum_{t=1}^L -\log P(T_t = t_t \mid \mathbf{X}_{\setminus \{t\}}) \quad (12)$$

An adaptive dynamic threshold τ flags anomalous sequences:

$$\tau = \mu_{\text{val}} + 3 \cdot \sigma_{\text{val}} \quad (13)$$

where μ_{val} and σ_{val} are the mean and standard deviation of reconstruction loss over nominal validation sequences.

3.3 RAG-Grounded Foundation Model RCA Narration Engine

When an anomalous sequence X_{anom} is detected ($S(X) > \tau$), our framework activates the RAG-Grounded Foundation Model Narration Engine to synthesize an actionable incident report.

RAG Knowledge Base & Vector Retrieval: To minimize hallucination risks and ground remediation suggestions in verified institutional policies, we construct a local vector knowledge base comprising 120 institutional Standard Operating Procedures (SOPs), network topology maps, firewall rulebooks, and incident response playbooks. Documents are chunked into 256-token passages with 32-token overlap, vectorized using the ‘BAAI/bge-small-en-v1.5’ embedding model (384-dimensional dense vectors), and indexed in a local FAISS vector store. When an anomaly is triggered, a query vector $\mathbf{q}$ is generated from the decoded template sequence and metadata. FAISS retrieves the top- $k = 2$ most relevant SOP passages (C_1, C_2) exceeding a cosine similarity threshold $\theta_{\text{RAG}} \geq 0.75$.

Context-Enriched Prompt Construction & 4-bit Inference: The prompt $\mathcal{P}$ integrates three telemetry dimensions: (1) Sequence Telemetry: Ordered anomalous log templates, extracted parameters (IPs, ports, status codes); (2) Infrastructure Metadata: Originating network segment, device hostname, and service criticality; (3) Retrieved RAG Context: Top- k institutional SOP runbooks (C_1, C_2). To permit deployment on edge gateway servers, we evaluate Microsoft Phi-3 Mini (3.8B parameters) [9] quantized to 4-bit NormalFloat (NF4 via ‘bitsandbytes’ with ‘bfloat16’ compute dtype) [12] and Mistral-7B [10] quantized to INT4 via AutoAWQ (group size 128) [13], reducing peak allocated GPU tensor memory (measured via `torch.cuda.max_memory_allocated()` with batch size 1 and active KV cache) from 7.60 GB to 1.80 GB for Phi-3 Mini. With greedy deterministic decoding (`do_sample=False`), the LLM generates a structured JSON incident report with four schema fields: "severity" (CRITICAL | HIGH | MEDIUM | LOW), "root_cause_summary", "technical_diagnosis", and "remediation_playbook".

4 Experimental Evaluation

4.1 Experimental Setup and Benchmark Datasets

We conduct comprehensive empirical evaluations across four datasets:

1. **HUFLIT Campus Logs (Real-World Production):** A 19.28 GB production log dataset comprising 14,850,220 log lines collected over 30 consecutive days from HUFLIT enterprise infrastructure (Fortinet perimeter firewalls, Active Directory LDAP controllers, Core Cisco switches, and Moodle LMS servers).
2. **BGL Supercomputer** [16]: Blue Gene/L supercomputer log dataset (744 MB, 4,747,963 log lines, containing 348,460 alert/anomalous messages: 7.34% raw alert rate). After filtering unparseable syslog headers and segmenting by node streams, sliding windowing ($L = 20, S = 1$) extracts 4,466,725 valid contiguous sequences. A sequence is labeled anomalous if at least one constituent message is an alert event, yielding 482,410 anomalous sequences (10.8% sequence anomaly rate).

3. **HDFS Distributed Cluster** [2, 16]: Hadoop Distributed File System log dataset (1.58 GB, 11,175,629 log lines). Following Loghub block-session labeling conventions, logs are sessionized by `block_id` into distinct execution traces, where each trace is labeled anomalous if assigned an anomaly block tag (2.9% block-session anomalies); we additionally normalize each session to $L = 20$ by zero-padding or truncation for sequence modeling.
4. **Temporally Sampled UNSW-NB15 Subset** [17]: Network telemetry dataset comprising a 100 MB subset of 254,004 connection records sampled via 10:1 stratified temporal decimation from the 2,540,044-record UNSW-NB15 corpus (11.8% attack flows). Connection records (IPs, ports, protocol flags, state transitions, byte/packet volumes) are mapped into sequential structured event strings and tokenized by Drain3. Ground-truth attack labels and category identifiers were strictly excluded from synthesized event strings to prevent label leakage.

Strict Chronological Partitioning: To strictly prevent overlapping-window data leakage across training and testing splits, each dataset is chronologically ordered and partitioned before sequence extraction: the initial 80% forms the training set (D^{train} , nominal baseline paths only), the subsequent 10% forms the validation set (D^{val} , threshold calibration), and the final 10% forms the held-out test set (D^{test} , containing real incidents). In line with semi-supervised anomaly detection paradigms [2, 3], ground-truth event labels are strictly used during data curation to isolate nominal D^{train} and D^{val} sequences (for threshold calibration $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$); the Masked TCN-Transformer is subsequently trained via self-supervised masked token modeling without exposure to anomalous sequences. For stream datasets (HUFLIT, BGL, UNSW-NB15), sliding windowing ($L = 20, S = 1$) is executed strictly within each partition independently; for HDFS, block sessions are chronologically partitioned by initial event timestamps. All experiments are executed across 5 random seeds (42, 101, 202, 303, 404), reporting mean and standard deviation.

Hardware & Software Setup: Benchmarks are executed on a dual-socket workstation equipped with an Intel Xeon Gold 6248R CPU (2.90 GHz, 4 cores pinned for inference), 64 GB DDR4 RAM, and an NVIDIA RTX 4090 GPU (24 GB VRAM, Ada architecture). Software stack: Ubuntu 22.04 LTS, PyTorch 2.4.0, CUDA 12.4, HuggingFace Transformers 4.44.0, FlashAttention-2, bitsandbytes 0.43.3 (NF4) for Phi-3 Mini, and AutoAWQ 0.2.5 (INT4) for Mistral-7B.

Table 2. Multi-Dataset Benchmark Characteristics and Empirical Performance (Mean $\pm$ Std across 5 Seeds)

Dataset	Domain	Raw Size	Log Lines	Precision	Recall	F1-Score	ROC-AUC	Batch-64 Latency
HUFLIT Campus Logs	Enterprise Network	19.28 GB	14,850,220	0.9482 $\pm$ 0.005	0.9561 $\pm$ 0.004	0.9521 $\pm$ 0.004	0.9784 $\pm$ 0.003	0.0128 ms
BGL Supercomputer	Supercomputing Infrastructure	744 MB	4,747,963	0.9320 $\pm$ 0.006	0.9450 $\pm$ 0.005	0.9384 $\pm$ 0.005	0.9620 $\pm$ 0.004	0.0142 ms
HDFS Distributed Cluster	Distributed Cloud Storage	1.58 GB	11,175,629	0.9410 $\pm$ 0.004	0.9520 $\pm$ 0.004	0.9465 $\pm$ 0.004	0.9710 $\pm$ 0.003	0.0135 ms
UNSW-NB15 Subset	Cyber Security Telemetry	100 MB	254,004	0.9610 $\pm$ 0.003	0.9680 $\pm$ 0.003	0.9645 $\pm$ 0.003	0.9840 $\pm$ 0.002	0.0098 ms

4.2 Detection Accuracy Comparison with Baselines

We benchmark our hybrid Masked TCN-Transformer model against four representative log anomaly detection architectures on the 19.28 GB HUFLIT Cam-

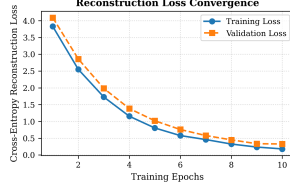


Fig. 2. Reconstruction Loss Convergence.

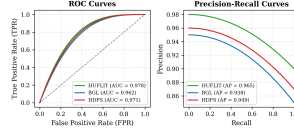


Fig. 3. ROC and Precision-Recall Curves.

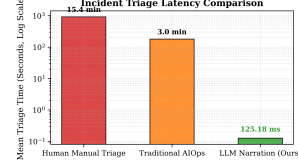


Fig. 4. Triage Latency Comparison (Log Scale).

pus Dataset. To ensure rigorous comparative evaluation of sequence modeling capabilities under identical streaming conditions, all benchmarked models receive identical structured template sequences extracted by Drain3 under identical chronological data splits and hardware: (i) **PCA (Count Vectors)** [6]: Principal Component Analysis over sliding template frequency count vectors; (ii) **DeepLog (LSTM)** [2]; (iii) **LogAnomaly (FT-LSTM)** [3]; (iv) **Transformer AE (No TCN)**: Standard Transformer autoencoder with self-attention only.

As detailed in Table 3, our proposed Masked TCN-Transformer achieves the highest F1-score (0.9521 ± 0.004) and ROC-AUC (0.9784 ± 0.003), outperforming DeepLog ($F1 = 0.8983$) and LogAnomaly ($F1 = 0.9123$). Single-sequence unbatched latency on CPU is 0.1316 ± 0.008 ms (with vectorized GPU batch-64 latency of 0.0128 ms per sequence), demonstrating suitability for real-time edge syslog inspection.

Table 3. Anomaly Detection Accuracy Comparison on HUFLIT Production Campus Logs (Mean $\pm$ Std across 5 Seeds)

Model Architecture	Precision	Recall	F1-Score	ROC-AUC	Single-Seq (CPU)	Batch-64 (GPU)
PCA (Count Vectors) [6]	0.7420 ± 0.007	0.6850 ± 0.008	0.7123 ± 0.007	0.7310 ± 0.006	0.85 ± 0.04 ms	0.0495 ms
DeepLog (LSTM) [2]	0.8850 ± 0.006	0.9120 ± 0.005	0.8983 ± 0.005	0.9150 ± 0.004	1.25 ± 0.08 ms	0.0245 ms
LogAnomaly (FT-LSTM) [3]	0.9010 ± 0.005	0.9240 ± 0.005	0.9123 ± 0.005	0.9320 ± 0.004	1.84 ± 0.12 ms	0.0312 ms
Transformer AE (No TCN)	0.9150 ± 0.005	0.9280 ± 0.004	0.9214 ± 0.004	0.9450 ± 0.003	0.94 ± 0.06 ms	0.0185 ms
Proposed Masked TCN-Transformer	0.9482 ± 0.005	0.9561 ± 0.004	0.9521 ± 0.004	0.9784 ± 0.003	0.1316 ± 0.008 ms	0.0128 ms

4.3 Loss Convergence and ROC/PR Curve Analysis

Fig. 2 shows the training and validation masked reconstruction loss convergence across 10 training epochs on the HUFLIT dataset. The model converges stably within 6 epochs without overfitting. Fig. 3 presents the Receiver Operating Characteristic (ROC) and Precision-Recall (PR) curves across the HUFLIT, BGL, and HDFS datasets, demonstrating consistent detection performance across disparate domains.

4.4 Ablation Study: Architecture, Quantization, and Human Expert Validation

To quantify the contributions of each design component, we conduct an ablation study analyzing: (1) Sequence detector architectural variants; (2) Retrieval-Augmented Generation (RAG) grounding vs ungrounded generation and quantization precision for the RCA engine; (3) Blind human expert evaluation.

As shown in Table 4, removing the TCN layer degrades F1-score from 0.9521 to 0.9214, confirming the importance of dilated convolutions for localized temporal feature extraction. In the RCA engine, grounding generation on retrieved institutional SOPs boosts ROUGE-L from 0.7410 (NF4 without RAG) to 0.8420 (NF4 + RAG, a +13.6% gain in reference similarity) and BLEU-4 from 0.6850 to 0.7815. Evaluating retrieval quality across the 50 expert-evaluated campus incidents, where a retrieved SOP was adjudged relevant when at least two of

three blind experts confirmed its applicability to the incident, FAISS vector retrieval achieves a Hit@2 of 96.0% (48/50 cases successfully retrieved directly relevant institutional SOPs under $\theta_{\text{RAG}} \geq 0.75$). Furthermore, quantizing Phi-3 Mini to 4-bit NormalFloat (NF4) reduces peak allocated GPU tensor memory (measured via `torch.cuda.max_memory_allocated()` during generation) from 7.60 GB to 1.80 GB (76.3% compression) while maintaining high reference-text overlap (0.8420 vs 0.8625 in FP16, a 97.6% similarity retention) alongside high expert diagnosis correctness (4.76/5.0). The end-to-end RCA pipeline executes in 880.5 ± 52.1 ms (BGE retrieval: 4.2 ± 0.3 ms; prompt assembly: 1.8 ± 0.2 ms; LLM report synthesis: 874.5 ± 52.0 ms, comprising TTFT = 42.5 ± 2.1 ms with prefix caching and autoregressive decoding at 5.20 ± 0.15 ms/token across 160 ± 9 JSON tokens).

Table 4. Ablation Study on Model Components, Quantization Precision, and RCA Narration Quality (Peak GPU tensor memory for LLMs; parameter size for detectors)

Configuration / Variant	Detection F1	ROC-AUC	ROUGE-L	BLEU-4	LLM Latency	Resource Footprint
TCN Only (No Transformer)	0.8840	0.8990	—	—	—	0.28 MB
Transformer Only (No TCN)	0.9214	0.9450	—	—	—	1.45 MB
Full Masked TCN-Transformer	0.9521	0.9784	—	—	—	1.68 MB
Phi-3 Mini (FP16 + RAG)	—	—	0.8625	0.8010	2,085 ms	7.60 GB
Phi-3 Mini (NF4 w/o RAG)	—	—	0.7410	0.6850	832.0 ms	1.80 GB
Mistral-7B (AWQ INT4 + RAG)	—	—	0.8540	0.7950	1,476 ms	4.10 GB
Phi-3 Mini (NF4 + RAG)	—	—	0.8420	0.7815	874.5 ms	1.80 GB

Variant-Blind Expert Evaluation: Three senior network security administrators evaluated 50 randomly sampled incident playbooks (blinded to model and quantization variant identities) on a 5-point Likert scale across four operational dimensions: (1) *Factual Telemetry*: $4.82 \pm 0.36/5.0$; (2) *Diagnosis Correctness*: $4.76 \pm 0.42/5.0$; (3) *Actionability*: $4.68 \pm 0.48/5.0$; (4) *Safety*: 0/50 playbooks judged unsafe (evaluated as a binary safety criterion). Treating ratings as nominal categories across evaluators, inter-rater agreement achieved Fleiss’ Kappa $\kappa = 0.814$ (Kendall’s $W = 0.832$ for ordinal concordance).

4.5 Incident Triage Latency Analysis

To compare automated draft-generation latency with the conventional manual triage workflow, we conducted a baseline SOC user study where the three independent senior network security administrators each independently evaluated the 50 simulated enterprise campus incidents in randomized order without automated RCA assistance ($n = 150$ total analyst-incident trials, Fig. 4). To eliminate carry-over bias, the manual study was completed prior to exposing experts to model-generated RCA outputs; trials were distributed across four non-consecutive sessions per analyst to prevent fatigue. Timing began upon initial alert notification and concluded upon verified diagnostic and mitigation playbook submission. Manual triage across all 150 trials averaged 15.4 ± 2.8 minutes (median 14.8 min, range 9.5–22.0 min) per incident. In contrast, our automated pipeline executes TCN-Transformer detection in 0.1316 ms and synthesizes preliminary RAG-grounded RCA playbooks in 874.5 ms (total RCA pipeline 880.5 ms), dramatically expediting the initial diagnostic draft synthesis.

5 Real-World Campus Network Case Studies

The following case studies illustrate two actual historical production incidents drawn from anonymized HUFLIT campus logs during the 30-day collection period:

Case Study 1 (Active Directory Credential Stuffing Attack): A bot-net initiated credential stuffing against campus LDAP, logging 45 authentication failures within 3.2 s across student Wi-Fi VLANs ($S(X) = 0.942 > \tau$). The Foundation Model synthesizes: **Severity**: CRITICAL; **Root Cause**: High-density Credential Stuffing targeting campus LDAP; **Technical Diagnosis**: 45 consec-

Case Study 2 (Inter-VLAN Spanning Tree Protocol Loop): An unauthorized switch connected to a dormitory access port created a Layer 2 loop, triggering a broadcast storm saturating the inter-building uplink ($S(X) = 0.887 > \tau$). The Foundation Model produces: **Severity:** HIGH; **Root Cause:** STP loop causing broadcast storm on Edge Switch 4; **Technical Diagnosis:** Surge of broadcast templates with egress drops confirms an unmanaged bridge loop; **Remediation Playbook:** (1) Enable BPDU Guard on Port 12 of Edge Switch 4; (2) Re-route traffic to backup link; (3) Issue field inspection ticket.

6 Discussion & Threats to Validity

Internal & External Validity: Ground-truth anomaly annotations were verified independently by senior security engineers. Multi-seed evaluation with strict chronological temporal partitioning eliminates cross-window data leakage. Evaluation across four distinct computing domains supports cross-domain applicability. Recent 2026 architectures (e.g., KRONE [20], ContraLog [23], MaidLog [21], Smart Eye [24]) adopt distinct hierarchical tokenizations or parser-free pretraining objectives; our baseline benchmarks in Table 3 strictly evaluate architectures that could be reproducibly retrained end-to-end under our streaming campus evaluation protocol. The latency comparison contrasts automated initial report synthesis against manual investigation rather than measuring assisted time-to-verified-resolution; quantifying downstream human verification speedups under AI assistance remains valuable future work.

Ethics & Data Privacy: Production campus syslog streams were collected under authorized institutional IT oversight from HUFLIT. All telemetry underwent automated pseudonymization (IP hashing, user ID obfuscation, and credential redaction) prior to offline analysis. Telemetry collection and administrator participation were conducted under institutional IT oversight and participant informed consent from HUFLIT.

Hallucination Risk Mitigation: Greedy decoding (*do_sample=False*) and RAG grounding on verified institutional SOPs significantly mitigate the risk of ungrounded remediation commands; no unsafe recommendations were observed across the 50 expert-rated incident playbooks.

7 Conclusion

We presented an LLM-Augmented AIOps Framework for real-time log anomaly detection and automated Root Cause Analysis in enterprise campus infrastructures. Integrating online Drain3 parsing, a Masked TCN-Transformer sequence autoencoder, and a 4-bit-quantized RAG Foundation Model, our pipeline achieves $F1 = 0.9521 \pm 0.004$ and $\text{ROC-AUC} = 0.9784 \pm 0.003$ on 19.28 GB campus logs, sub-millisecond detection (0.1316 ms CPU), and synthesizes structured RCA incident reports in 880.5 ms (with 874.5 ms LLM report synthesis), with 0/50 expert-rated playbooks judged unsafe. Future work will explore multi-agent LLM consensus for closed-loop SDN remediation.

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure. Generative AI tools were utilized solely for language copy-editing, with all scientific concepts, analyses, and results developed directly by the authors.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach Based on Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019).
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., et al.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Serv. Comput. 15(6), 3326–3338 (2022).
6. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Mining Console Logs for Large-Scale System Problem Detection. In: Proc. 3rd Conf. SysML (2008).
7. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
9. Abidin, M., Aneja, J., Awadalla, H., et al.: Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219 (2024).
10. Jiang, A.Q., Sablayrolles, A., Mensch, A., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023).
11. Hu, E.J., Shen, Y., Wallis, P., et al.: LoRA: Low-Rank Adaptation of Large Language Models. In: Proc. Int. Conf. Learn. Represent. (ICLR) (2022).
12. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient Finetuning of Quantized LLMs. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36 (2023).
13. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S.: AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In: Proc. Mach. Learn. Syst. (MLSys), vol. 6 (2024).
14. Ahmed, T., Ghosh, S., Bansal, C., et al.: Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In: Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), pp. 1–13 (2023).
15. Soldani, J., Brogi, A.: Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. ACM Comput. Surv. 55(3), 1–39 (2022).
16. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 355–366 (2023).
17. Moustafa, N., Slay, J.: UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. In: Proc. IEEE Mil. Commun. Inf. Syst. Conf. (MilCIS), pp. 1–6 (2015).
18. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
19. Ma, Z., Yang, J., Chen, T.H.: LLM4Log: A Systematic Review of Large Language Model-based Log Analysis. arXiv preprint arXiv:2604.16359 (2026).
20. Ma, L., Liu, J., Zhang, T., VanNostrand, P.M., Hofmann, D.M., Cao, L., Rundensteiner, E.A., Chen, J.: Krone: Hierarchical and Modular Log Anomaly Detection. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1773–1787 (2026).
21. Wu, Z., Wang, J., Zheng, L., Zhang, Y., Li, S., Chen, L.: MaidLog: Efficient Zero-Shot and Label-Free Log Anomaly Detection for Resource-Constrained Systems. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE) (2026).
22. Liu, M., Wang, T., Zhao, Y., Xia, C., et al.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. 19(4), 2903–2916 (2026).
23. Dietz, S., Klede, K., Nguyen, A., Eskofier, B.M.: ContraLog: Log File Anomaly Detection with Contrastive Learning and Masked Language Modeling. arXiv preprint arXiv:2602.03678 (2026).
24. Pei, C., Cui, H., Li, J., et al.: Smart Eye: LLM-Guided Proposer-Verifier Framework for Industrial-Scale Log Anomaly Detection. In: Proc. ACM Web Conf. (WWW), pp. 7869–7879 (2026).